

OBJECT-ORIENTED PROGRAMMING

LECTURE OUTLINE

In this lecture, we will cover the following topics:

- What is Object Orientation?
- Classes and Thereabouts
- SOLID Principles
- Type Systems, Inheritance and Polymorphism
- Design Patterns

OBJECT ORIENTATION

The observation, in the practice of software engineering, that data and logic pertaining to it are often localised and highly cohesive, with natural hierarchical structures and loose interactions across units.

PS: The jury is still out on whether this remains true in the age of deep learning models.

CLASSES AND THEREABOUTS

	Attribute Declaration	Constructor Implementation	Method Declaration	Method Implementation
Object Literals	✓	✗	✗	✗
Pure Dataclasses	✓	✓	✗	✗
Interfaces / Traits	?	✗	✓	✗
Object Literal Types	✓	✗	✓	✗
Mixins	✗	✗	✓	✓
Abstract Classes	✓	?	✓	?
Classes	✓	✓	✓	✓

SOLID PRINCIPLES

SOLID is an acronym for five historical principles that have shaped the development of object orientation as a software engineering paradigm:

- Single-Responsibility Principle
- Open/Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

SINGLE-RESPONSIBILITY PRINCIPLE

“Each software unit should have one, and only one, responsibility.”

The following OOP concepts are closely related to this principle:

- Loose coupling + high cohesion (design).
- Encapsulation (design).
- Accessibility Modifiers (language feature).
- Properties, Descriptors (language features).

OPEN-CLOSED PRINCIPLE

“Software units should be open for extension but closed for modification.”

The following OOP concepts are closely related to this principle:

- Inheritance, Polymorphism (class-level reuse).
- Packages, Import Systems (module-level reuse).
- Package Managers, Package Repositories (library-level reuse).

LISKOV SUBSTITUTION PRINCIPLE

“A type S is a subtype of T if an object of type S can be used wherever an object of type T is expected without altering correctness of the program.”

This principle is the foundation of all modern OOP type systems.

INTERFACE SEGREGATION PRINCIPLE

“No code should be force to depend on methods it does not use.”

The following OOP concepts are closely related to this principle:

- Interfaces, Traits (shape dependency).
- Mixins (logic dependency).
- Multiple Inheritance (composition).

DEPENDENCY INVERSION PRINCIPLE

“High-level software units should not depend directly on low-level ones.”

The following OOP concepts are closely related to this principle:

- Interfaces (language feature).
- Loose coupling (design).
- Dependency Injection (practice).

TYPE SYSTEMS

Three main typing philosophies in OOP:

- Nominal Typing: e.g. Java, C#, Rust, C++, Julia
- Structural Typing: e.g. TypeScript, OCaml, Scala, Go
- Duck Typing: e.g. JavaScript, Ruby, Smalltalk

Python supports all three in various forms.

INHERITANCE

Inheritance = Shape and logic reuse in OOP subtypes.

Progressively more sophisticated inheritance mechanisms:

- No Inheritance: e.g. JavaScript
- Interface/Trait Implementation Only: e.g. Rust, Go, Zig
- Single Inheritance: e.g. Java 7-, C# 7-
- Mixins: e.g. Scala, Kotlin, Java 8+, C# 8+
- Multiple Inheritance: e.g. C++, Python

POLYMORPHISM

Polymorphism = The same software unit used under different shapes.

Important instances of polymorphism in OOP:

- Inheritance, Subtyping
- Function Overloading (aka ad-hoc polymorphism)
- Parametric Polymorphism (aka type-level generics)
- Templates (aka monomorphised generics)

DESIGN PATTERNS

Some design patterns have been internalised into OOP language features:

- Decorator
- Iterator
- Resource Acquisition is Initialisation (RAII)
- Singleton (e.g. module imports)
- Publish-Subscribe
- Fluent API

DESIGN PATTERNS

Other design patterns remain less standardised. Examples include:

- Facade
- Factory
- Builder
- Flyweight
- State
- Visitor
- Memento